

1. Plan de test unitaire

1.1. Identification du test

Ihm.EnvoiePointDeMesure_v1

1.2. Référence au module testé

Méthode testé : string EnvoiePointDeMesure(List<PointLatLng> points_mesure)

Cette méthode appartient à la classe Ihm et aura comme possible valeur de retour :

Valeur de retour	Description
résultat	le résultat de l'envoi des points de mesure

1.3. Objectif du test

Ce test aura pour but de tester l'envoi des points de mesures via la méthode EnvoiePointDeMesure en y insérant plusieurs points de mesure fictif correspondant au cas de test cité dans la partie 1.4.1

1.4. Procédure de test

- Ouverture du projet de test
- Configuration des vecteurs de test en fonction de la méthode de test (les méthodes de test ont le même nom que le cas de test)
- Lancement du projet
- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

Plan de tests unitaires méthode *EnvoiePointDeMesure*

1.4.1. Valeurs de test :

Nom du test	Vecteur de test
TC_01	points_mesure[0] = (0;0)
TC_02	points_mesure[0] = (47.543; -0.5124)
TC_03	points_mesure[0] = (90; -180)
TC_04	points_mesure[0] = (-90; 180)
TC_05	points_mesure[0] = (-108.567; -0.5432)
TC_06	points_mesure[0] = (47.546; 198.678)
TC_07	points_mesure[0] = ("Mauvais Format"; -0.5467)
TC_08	points_mesure[0] = (47.567;"Mauvais Format")
TC_09	points_mesure[0] = (47.567896543, -0.54325725)
TC_10	points_mesure[0] = "Mauvais Format"
TC_11	points_mesure = {(48.5678; -0.4967), (47.543; -0.5124)}
TC_12	point_mesure = {(47.543; -0.5124); (98.567; 187.456)}
TC_13	point_mesure = {(98.567; 187.456),(47.543; -0.5124)}
TC_14	point_mesure = {(47.543; -0.5124), (47.543; -0.5124)}
TC_15	point_mesure = null
TC_16	point_mesure = []

1.4.2. TC_01 : Cas Valide : Origine du repère

a. Initialisation

points_mesure contient une coordonnées GPS de coordonnées (0;0)

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout d'un PointLatLng de coordonnées (0;0) dans points_mesure
- Ajout d'un Assert.AreEquals vérifiant que la méthode renvoie bien un message disant "les points de mesures ont bien été envoyé"
- Lancement du projet

Plan de tests unitaires méthode EnvoiePointDeMesure

- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un **Assert.AreEqual()**

la méthode est valide si le Assert.AreEqual() **ne retourne pas d'exception**

1.4.3. TC_02 : Cas Valide : Point lambda

a. Initialisation

points_mesure contient **une coordonnées GPS de coordonnées 47.543; -0.5124)**

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout d'un PointLatLng de coordonnées (47.543; -0.5124) dans points_mesure
- Ajout d'un Assert.AreEquals vérifiant que la méthode renvoie bien un message disant "les points de mesures ont bien été envoyé"
- Lancement du projet
- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un **Assert.AreEqual()**

la méthode est valide si le Assert.AreEqual() **ne retourne pas d'exception**

1.4.4. TC_03 : Cas limite 1

a. Initialisation

points_mesure contient une coordonnées GPS de coordonnées (90 ; -180)

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout d'un PointLatLng de coordonnées (90; -180) dans points_mesure
- Ajout d'un Assert.AreEquals vérifiant que la méthode renvoie bien un message disant "les points de mesures ont bien été envoyé"
- Lancement du projet
- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un **Assert.AreEqual()**

la méthode est valide si le Assert.AreEqual() **ne retourne pas d'exception**

1.4.5. TC_04 : Cas limite 2

a. Initialisation

points_mesure contient une coordonnées GPS de coordonnées (-90 ; 180)

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout d'un PointLatLng de coordonnées (-90; 180) dans points_mesure
- Ajout d'un Assert.AreEquals vérifiant que la méthode renvoie bien un message disant "les points de mesures ont bien été envoyé"

Plan de tests unitaires méthode EnvoiePointDeMesure

- Lancement du projet
- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un **Assert.AreEqual()**

la méthode est valide si le Assert.AreEqual() **ne retourne pas d'exception**

1.4.6. TC_05 : Cas Invalide : latitude invalide

a. Initialisation

points_mesure contient **une coordonnées GPS de coordonnées (-108.567 ; -0.5432)**

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout d'un PointLatLng de coordonnées (-108.567; -0.5432) dans points_mesure
- Ajout d'un Assert.AreEquals vérifiant que la méthode lève bien une exception de type ArgumentOutOfRangeException
- Lancement du projet
- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un **Assert.ThrowsException<>()**

la méthode est valide si le Assert.ThrowsException retourne une exception de type **ArgumentOutOfRangeException**

Plan de tests unitaires méthode EnvoiePointDeMesure

1.4.7. TC_06 : Cas Invalide : Longitude invalide

a. Initialisation

points_mesure contient une coordonnées GPS de coordonnées (47.546; 198.678)

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout d'un PointLatLng de coordonnées (47.546; 198.678) dans points_mesure
- Ajout d'un Assert.ThrowsException vérifiant que la méthode lève bien une exception de type ArgumentOutOfRangeException
- Lancement du projet
- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un

Assert.ThrowsException<>()

la méthode est valide si le Assert.ThrowsException retourne une exception de type ArgumentOutOfRangeException

1.4.8. TC_07 : Cas Invalide : Latitude dans le mauvais format

a. Initialisation

points_mesure contient une coordonnées GPS de coordonnées ("Mauvais Format", -0.5467)

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout d'un PointLatLng de coordonnées ("Mauvais Format", -0.5467) dans points_mesure
- Ajout d'un Assert.ThrowsException vérifiant que la méthode lève bien une exception de type ArgumentException
- Lancement du projet

Plan de tests unitaires méthode EnvoiePointDeMesure

- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un

Assert.ThrowsException<>()

la méthode est valide si le Assert.ThrowsException retourne une exception de type **ArgumentException**

1.4.9. TC_08 : Cas Invalide : Longitude dans le mauvais format

a. Initialisation

points_mesure contient **une coordonnées GPS de coordonnées (47.567; "Mauvais Format")**

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout d'un PointLatLng de coordonnées (47.567; "Mauvais Format") dans points_mesure
- Ajout d'un Assert.ThrowsException vérifiant que la méthode lève bien une exception de type ArgumentException
- Lancement du projet
- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un

Assert.ThrowsException<>()

la méthode est valide si le Assert.ThrowsException retourne une exception de type **ArgumentException**

Plan de tests unitaires méthode EnvoiePointDeMesure

1.4.10. TC_09 : Cas valide : test arrondi

**La méthode arrondi à 5 décimal donc ce cas de test permettra de tester si l'arrondissement de la valeur se passe bien*

a. Initialisation

points_mesure contient une coordonnées GPS de coordonnées (47.567896543; -0.54325725)

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout d'un PointLatLng de coordonnées (47.567896543; -0.54325725) dans points_mesure
- Ajout d'un Assert.AreEquals vérifiant que la méthode renvoie bien un message disant "les points de mesures ont bien été envoyé"
- Lancement du projet
- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un Assert.AreEqual()

la méthode est valide si le Assert.AreEqual() ne retourne pas d'exception

1.4.11. TC_10 : Cas invalide : Point dans le mauvais format

a. Initialisation

points_mesure contient un string contenant "Mauvais Format"

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout d'un PointLatLng de coordonnées "Mauvais Format" dans points_mesure
- Ajout d'un Assert.ThrowsException vérifiant que la méthode lève bien une exception de type ArgumentException

Plan de tests unitaires méthode EnvoiePointDeMesure

- Lancement du projet
- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un

Assert.ThrowsException<>()

la méthode est valide si le Assert.ThrowsException retourne une exception de type **ArgumentException**

1.4.12. TC_11 : Cas Valide : 2 Point lambda

a. Initialisation

points_mesure contient **deux coordonnées GPS de coordonnées (48.5678; -0.4967) et (47.543; -0.5124)**

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout de deux PointLatLng de coordonnées (48.5678; -0.4967) et (47.543; -0.5124) dans points_mesure
- Ajout d'un Assert.AreEquals vérifiant que la méthode renvoie bien un message disant "les points de mesures ont bien été envoyé"
- Lancement du projet
- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un **Assert.AreEqual()**

la méthode est valide si le Assert.AreEqual() **ne retourne pas d'exception**

Plan de tests unitaires méthode *EnvoiePointDeMesure*

1.4.13. TC_12 : Cas Invalide : deux point dont un hors limite

a. Initialisation

points_mesure contient deux coordonnées GPS de coordonnées (47.543; -0.5124) et (98.567; 187.456)

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout de deux PointLatLng de coordonnées (47.543; -0.5124) et (98.567; 187.456) dans points_mesure
- Ajout d'un Assert.ThrowsException vérifiant que la méthode lève bien une exception de type ArgumentOutOfRangeException
- Lancement du projet
- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un

Assert.ThrowsException<>()

la méthode est valide si le Assert.ThrowsException retourne une exception de type ArgumentOutOfRangeException

1.4.14. TC_13 : Cas Invalide : Inverse du TC_12

a. Initialisation

points_mesure contient deux coordonnées GPS de coordonnées (98.567; 187.456) et (47.543; -0.5124)

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout de deux PointLatLng de coordonnées (98.567; 187.456) et (47.543; -0.5124) dans points_mesure
- Ajout d'un Assert.ThrowsException vérifiant que la méthode lève bien une exception de type ArgumentOutOfRangeException
- Lancement du projet

Plan de tests unitaires méthode EnvoiePointDeMesure

- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un

Assert.ThrowsException<>()

la méthode est valide si le Assert.ThrowsException retourne une exception de type **ArgumentOutOfRangeException**

1.4.15. TC_14 : Cas Invalide : deux point identique

a. Initialisation

points_mesure contient **deux coordonnées GPS de coordonnées (47.543; -0.5124) et (47.543; -0.5124)**

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout de deux PointLatLng de (47.543; -0.5124) et (47.543; -0.5124) dans points_mesure
- Ajout d'un Assert.ThrowsException vérifiant que la méthode lève une exception de type ArgumentException
- Lancement du projet
- Comparaison des résultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un

Assert.ThrowsException<>()

la méthode est valide si le Assert.ThrowsException retourne une exception de type **ArgumentException**

1.4.16. TC_15 : Cas invalide : Liste Null

a. Initialisation

points_mesure est null

b. Lancement

- Ouverture du projet de test
- Initialisation de point de mesure comme etant null
- Ajout d'un Assert.ThrowsException verifiant que la methode leve bien une exception de type ArgumentNullException
- Lancement du projet
- Comparaison des resultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un

Assert.ThrowsException<>()

la méthode est valide si le Assert.ThrowsException retourne une exception de type ArgumentNullException

1.4.17. TC_16 : Cas invalide : Liste vide

a. Initialisation

points_mesure est vide

b. Lancement

- Ouverture du projet de test
- Vidage des points stocké dans point de mesure
- Ajout d'un Assert.ThrowsException vérifiant que la méthode leve bien une exception de type ArgumentNullException
- Lancement du projet
- Comparaison des resultat obtenu avec les résultats attendu (partie 1.5 du plan de test)
- Validation du cas de test

Plan de tests unitaires méthode EnvoiePointDeMesure

c. Observation des résultats

Pour observer le résultat on compare la valeur retournée par la méthode avec la valeur attendu a l'aide d'un

Assert.ThrowException<>()

la méthode est valide si le Assert.ThrowException retourne une exception de type **ArgumentNullException**

1.5. Résultat attendu

Paramètre d'entrée	Résultat attendu
points_mesure[0] = (0;0)	La méthode retourne un message disant que les points de mesure ont bien été envoyé
points_mesure[0] = (47.543; -0.5124)	La méthode retourne un message disant que les points de mesure ont bien été envoyé
points_mesure[0] = (90; -180)	La méthode retourne un message disant que les points de mesure ont bien été envoyé
points_mesure[0] = (-90; 180)	La méthode retourne un message disant que les points de mesure ont bien été envoyé
points_mesure[0] = (-108.567; -0.5432)	La méthode lève une exception de type ArgumentOutOfRangeException
points_mesure[0] = (47.546; 198.678)	La méthode lève une exception de type ArgumentOutOfRangeException
points_mesure[0] = ("Mauvais Format"; -0.5467)	La méthode lève une exception de type ArgumentException
points_mesure[0] = (47.567;"Mauvais Format")	La méthode lève une exception de type ArgumentException
points_mesure[0] = (47.567896543, -0.54325725)	La méthode arrondi les coordonnées GPS à 5 décimal et retourne un message disant que les points de mesure ont bien été envoyé
points_mesure[0] = "Mauvais Format"	La méthode lève une exception de type ArgumentException
points_mesure = {(48.5678; -0.4967), (47.543; -0.5124)}	La méthode retourne un message disant que les points de mesure ont bien été envoyé
point_mesure = {(47.543; -0.5124); (98.567; 187.456)}	La méthode lève une exception de type ArgumentOutOfRangeException
point_mesure = {(98.567; 187.456),(47.543; -0.5124)}	La méthode lève une exception de type ArgumentOutOfRangeException
point_mesure = {(47.543; -0.5124), (47.543; -0.5124)}	La méthode lève une exception de type ArgumentException
point_mesure = null	La méthode lève une exception de type ArgumentNullException
point_mesure = []	La méthode lève une exception de type ArgumentNullException

Plan de tests unitaires méthode EnvoiePointDeMesure

1.6. Moyen à mettre en oeuvre

Moyen nécessaire pour mettre en oeuvre ce test :

- Un PC fixe ou portable équipé d'un écran avec Windows 10 ou 11 installé dessus
- Visual studio configurer pour pouvoir compiler et exécuter des solutions utilisant le framework .NET et MS Test
- La solution téléchargé sur le PC
- Une carte avec un code de réception de données via une communication TCP permettant de simuler l'obtention des points par le robot ou le robot lui même

2. Procès Verbal

Élément testé	Ihm.EnvoiePointDeMesure			
Objectif du test	Vérification du bonne envoie des points de mesure au robot			
Nom du testeur	BELAIRE Kévin		Date	24/04/2026
Moyen mis en oeuvre	Logiciel : Visual Studio 2022	Matériel : PC		Outil de développement : Framework .NET
Procédure du test : -				
Id	Description du vecteur test	Résultat attendu	Résultat obtenu	Validation (O/N)
TC_01	points_mesure[0] = (0;0)	La methode retourne un message disant que les points de mesure ont bien été envoyé		
TC_02	points_mesure[0] = (47.543; -0.5124)	La methode retourne un message disant que les points de mesure ont bien été envoyé		
TC_03	points_mesure[0] = (90; -180)	La methode retourne un message disant que les points de mesure ont bien été envoyé		
TC_04	points_mesure[0] = (-90; 180)	La methode retourne un message disant que les points de mesure ont bien été envoyé		
TC_05	points_mesure[0] = (-108.567; -0.5432)	La methode leve une exception de type ArgumentOutOfRangeException		
TC_06	points_mesure[0] = (47.546; 198.678)	La methode leve une		

Plan de tests unitaires méthode EnvoiePointDeMesure

		exception de type ArgumentOutOfRangeException		
TC_07	points_mesure[0] = ("Mauvais Format"; -0.5467)	La methode leve une exception de type ArgumentException		
TC_08	points_mesure[0] = (47.567;"Mauvais Format")	La methode leve une exception de type ArgumentException		
TC_09	points_mesure[0] = (47.567896543, -0.54325725)	La méthode arrondi les coordonnée GPS à 5 décimal et retourne un message disant que les points de mesure ont bien été envoyé		
TC_10	points_mesure[0] = "Mauvais Format"	La methode leve une exception de type ArgumentException		
TC_11	points_mesure = {(48.5678; -0.4967), (47.543; -0.5124)}	La methode retourne un message disant que les points de mesure ont bien été envoyé		
TC_12	point_mesure = {(47.543; -0.5124); (98.567; 187.456)}	La methode leve une exception de type ArgumentOutOfRangeException		
TC_13	point_mesure = {(98.567; 187.456),(47.543; -0.5124)}	La methode leve une exception de type ArgumentOutOfRangeException		
TC_14	point_mesure = {(47.543; -0.5124), (47.543; -0.5124)}	La methode leve une exception de type ArgumentException		
TC_15	point_mesure = null	La methode leve une exception de type ArgumentNullException		
TC_16	point_mesure = []	La methode leve une exception de type ArgumentNullException		